

Informe Técnico Profesional

Fecha: 2026-08-12

Autores:

- Marcela Baldeón
- Rosa Samaniego

Índice

- Resumen ejecutivo
- 1. Objetivos
- 2. Alcance
- 3. Diseño y arquitectura
- 3.1 Componentes principales
- 3.2 Modelo de datos (resumen)
- 4. Validaciones y reglas de negocio
- 5. Calidad del código y análisis estático
- 6. Pruebas
- 7. Instrucciones de uso y verificación
- 8. Resultados y evidencias
- 9. Conclusiones
- 10. Recomendaciones técnicas
- 11. Anexos

Resumen ejecutivo

Este documento presenta el Informe Técnico Profesional del proyecto "Caso Integrador Final", cuyo objetivo fue desarrollar un sistema de gestión de capacitación en Python capaz de registrar estudiantes, controlar asistencia por módulos, gestionar actividades parciales y administrar la habilitación y registro de examen final. El sistema incluye validaciones robustas, persistencia en formato JSON, una interfaz de consola para interacción y una batería de pruebas automatizadas que verifican la correcta operación del conjunto.

1. Objetivos

- Desarrollar una aplicación que permita la gestión integral de la inscripción, asistencia y evaluación de estudiantes en cursos de capacitación.
- Implementar validaciones formales (especialmente validación de cédula ecuatoriana) para asegurar la integridad de los datos.
- Proveer persistencia y recuperación de datos mediante archivos JSON.
- Desarrollar pruebas automatizadas que garanticen la estabilidad funcional del sistema.

2. Alcance

El sistema cubre:

- Registro de estudiantes con validación de identidad.
- Registro de asistencia por módulo, con control de horas por entrada y límites establecidos.
- Registro de dos actividades parciales por módulo (30 puntos cada una) y un examen final (20 puntos).
- Reglas de habilitación para el examen final basadas en asistencia mínima y puntajes de actividades.
- Persistencia local en `estudiantes.json` y carga segura desde ficheros, omitiendo registros con cédulas inválidas.
- Interfaz de consola básica para uso interactivo y mecanismos de prueba automatizada.

3. Diseño y arquitectura

3.1. Componentes principales

- `sistema\_capacitacion.py`: módulo principal que implementa los modelos de datos, las operaciones del sistema (registro, asistencia, actividades, evaluación, persistencia) y funciones utilitarias de validación.

- Tests: Suites basadas en `unittest` y `pytest` para pruebas unitarias e integración.
- Configuración de pruebas: `pytest.ini` añadido en la raíz y una copia en `caso\_integrador\_final` para permitir ejecución desde ese subdirectorio.

3.2. Modelo de datos (resumen)

- Entidades: `Estudiante`, `Modulo`, `ActividadParcial` (implementadas mediante `dataclass` en Python).
- Persistencia: archivo JSON con mapa de identificaciones a objetos serializados.

4. Validaciones y reglas de negocio

- Validación de cédula ecuatoriana: comprobación de formato (10 dígitos numéricos) y verificación del dígito verificador. Se implementó una función con mensaje detallado de motivo cuando la validación falla.
- Límite por entrada de asistencia: las inserciones que intenten registrar más de 40 horas por entrada son rechazadas.
- Control de habilitación: el sistema verifica asistencia mínima y puntajes de actividades antes de permitir el registro del examen final.

5. Calidad del código y análisis estático

- Se aplicaron refactorizaciones para reducir duplicación de literales y complejidad cognitiva en funciones críticas.
- Se documentaron y mitigaron advertencias de herramientas estáticas a través de cambios locales y exclusiones para archivos de terceros (p. ej. `typeshed` de Pylance). Para evitar falsos positivos en Sonar se añadió `sonar-project.properties` con exclusiones específicas y una configuración en `.vscode/settings.json` para ocultar rutas de extensiones en la vista del editor.

6. Pruebas

- Frameworks: `unittest` y `pytest`.
- Configuración: `pytest.ini` en la raíz y en `caso\_integrador\_final` para garantizar descubrimiento de pruebas incluso cuando se ejecuta desde el subdirectorio.
- Resultado de la ejecución (verificación realizada el 2026-08-12):
- Total de pruebas: 32
- Estado: 32 passed

7. Instrucciones de uso y verificación

Para ejecutar las pruebas desde la carpeta `caso\_integrador\_final` (recomendado por la presente entrega):

```
```bash
cd caso_integrador_final
python -m pytest -v
```
```

Para ejecutar la aplicación interactiva (mostrar menú):

```
```bash
python ../sistema_capacitacion.py
```
```

8. Resultados y evidencias

- La batería de pruebas automatizadas se ejecutó satisfactoriamente y todas las pruebas pasaron.
- La validación de cédulas fue probada con generadores deterministas en los tests y con casos de formato inválido para asegurar la robustez.

9. Conclusiones

El sistema alcanzó los objetivos planteados: maneja inscripciones, control de asistencia, actividades y examen final con reglas de habilitación claras y verificables. La inclusión de validaciones y pruebas

automatizadas aumenta la confianza en la integridad de los datos y la estabilidad funcional del proyecto.

10. Recomendaciones técnicas

- Introducir control de concurrencia o una base de datos ligera (p. ej. SQLite) si se requiere acceso concurrente o mayores volúmenes de datos.
- Añadir integración continua que ejecute `pytest` y un análisis de Sonar en cada `push` para mantener la calidad del código.
- Documentar contratos de la API interna (si se abre a otros módulos) y añadir más pruebas de borde para rutas de error.

11. Anexos

- Archivos relevantes:
- `sistema\_capacitacion.py` (implementación principal)
- `test\_pytest\_sistema.py`, `test\_sistema.py`, `test\_modulo.py` (pruebas)
- `caso\_integrador\_final/pytest.ini` (configuración local de pytest)
- `sonar-project.properties` (exclusiones para Sonar)
- Comandos útiles:

```bash

## Ejecutar pruebas desde el subdirectorio

```
cd caso_integrador_final
python -m pytest -v
```

## Ejecutar módulo principal (desde caso\_integrador\_final)

```
python ../sistema_capacitacion.py
```

```

Este informe fue generado como entrega técnica del trabajo realizado para el `Caso Integrador Final` y resume las decisiones de diseño, las validaciones implementadas, los resultados de prueba y las recomendaciones para continuidad y mantenimiento.